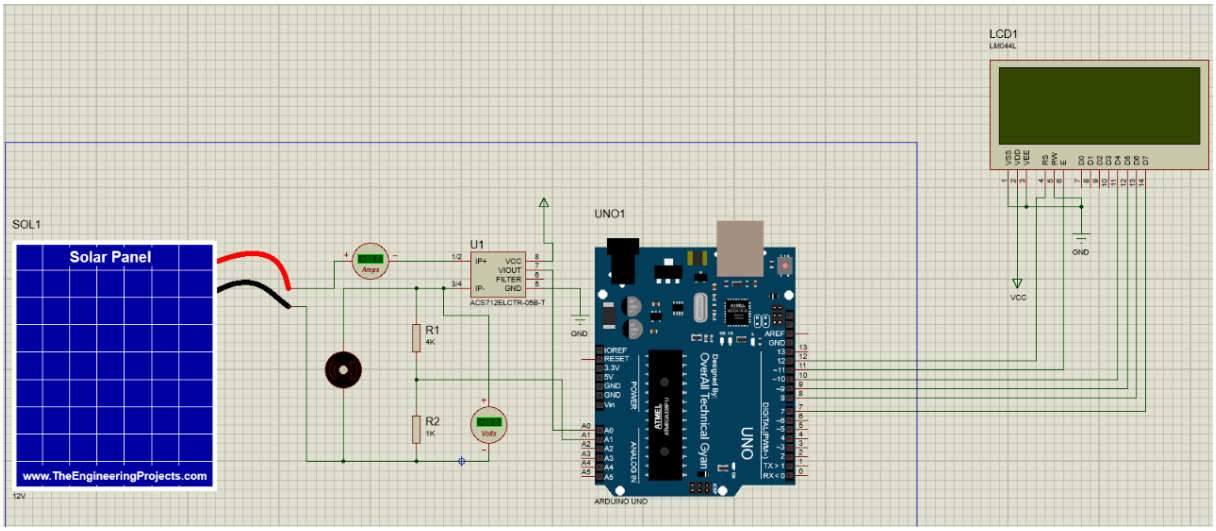
LABORATORY MANUAL

Design and Implementation of an Arduino-Based Solar Monitoring System

Using ACS712 Current Sensor, Voltage Divider, and LM044L 20×4 LCD



Experiment Information

Item	Description
Experiment title	Design and Implementation of an Arduino-Based Solar Monitoring System
Platform	Arduino UNO and Proteus Design Suite
Display	LM044L 20×4 character LCD
Sensors	ACS712ELCTR-05B-T (5A) current sensor and resistor voltage divider
Programming language	Arduino C/C++

1. Objectives

- Understand the basic operation of the Arduino UNO microcontroller.
- Interface an LM044L 20×4 LCD with Arduino UNO.
- Measure solar-panel voltage using a resistor voltage divider.
- Measure load current using the ACS712 Hall-effect current sensor.
- Calculate electrical power from measured voltage and current.
- Display voltage, current, and power on the LCD.
- Build and test the circuit using Proteus Design Suite.

2. Required Components and Software

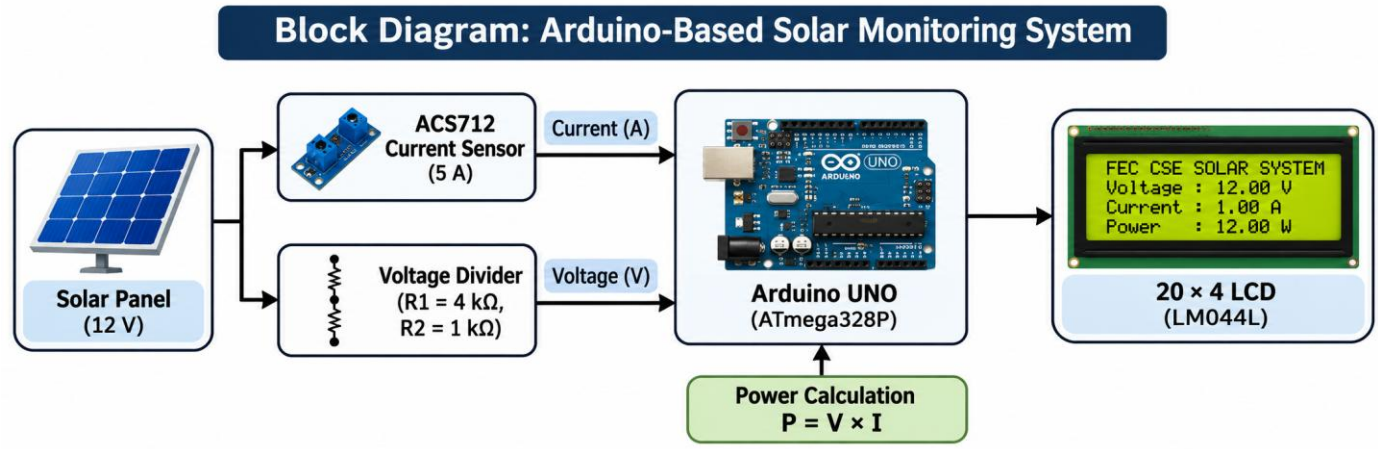
No.	Component / Software	Specification	Qty.
1	Arduino UNO	ATmega328P, 16 MHz	1
2	Solar panel source	12 V nominal (Proteus model)	1
3	Current sensor	ACS712ELCTR-05B-T, ±5 A	1
4	LCD	LM044L, 20 columns × 4 rows	1
5	Resistor R1	4 kΩ	1
6	Resistor R2	1 kΩ	1
7	DC motor / load	Suitable DC load	1
8	Ammeter and voltmeter	For reference observation	As needed
9	Software	Arduino IDE and Proteus Design Suite	—

3. Introduction

A solar monitoring system measures electrical parameters produced by a solar panel. In this experiment, Arduino UNO reads the panel voltage through a resistor voltage divider and reads the load current through an ACS712 current sensor. The microcontroller calculates electrical power and displays the measured values on an LM044L 20×4 LCD.

The measured quantities are solar-panel voltage (V), current (A), and calculated power (W). The system is intended for educational simulation and demonstrates analog interfacing, sensor interfacing, ADC conversion, LCD control, and basic embedded-system programming.

4. System Block Diagram



The voltage divider and ACS712 provide analog signals to the Arduino. The Arduino converts these signals into engineering units, calculates power, and updates the LCD.

5. Circuit Description and Theory

5.1 Solar panel

The solar panel converts light energy into electrical energy. Its output depends on illumination, panel characteristics, and the connected load. The 12 V label in the Proteus schematic is a nominal source description; actual voltage and current depend on the simulated model and operating point.

5.2 Voltage measurement using a voltage divider

The Arduino UNO analog input should remain within its permitted input range (normally 0–5 V when using the default 5 V reference). The panel voltage is reduced by a 4 kΩ / 1 kΩ divider before it reaches A0.

Voltage-divider equation:

$V_{A0} = V_{PANEL} \times R2 / (R1 + R2)$

$V_{PANEL} = V_{A0} \times (R1 + R2) / R2 = V_{A0} \times 5$

For R1 = 4 kΩ and R2 = 1 kΩ, the voltage at A0 is one-fifth of the panel voltage. For example, 12 V at the panel produces approximately 2.4 V at A0.

5.3 Current measurement using ACS712

The ACS712 is a Hall-effect current sensor. Current flows through its IP+ and IP− terminals, while its VIOUT pin provides an analog voltage to the Arduino. The ACS712ELCTR-05B-T is the 5 A version, with nominal sensitivity of 185 mV/A (0.185 V/A). Its zero-current output is approximately half the supply voltage, nominally 2.5 V when powered from 5 V.

$I = (VIOUT - VZERO) / 0.185$

The zero-current voltage varies with the sensor and model. For accurate readings, measure or calibrate VZERO with no current flowing. The example code uses 2.5 V as a nominal starting value.

5.4 Power calculation

$P = V \times I$

Power is expressed in watts when voltage is in volts and current is in amperes. For example, 12 V × 1 A = 12 W.

6. Detailed Pin Connections

6.1 LM044L 20×4 LCD to Arduino UNO

LCD pin	LCD signal	Arduino / supply connection	Purpose
1	VSS	GND	Ground
2	VDD	+5 V	LCD supply
3	VEE / VO	GND for basic test, or potentiometer wiper	Contrast adjustment
4	RS	D12	Register select
5	RW	GND	Write-only operation
6	E	D11	Enable
7–10	D0–D3	Not connected	Unused in 4-bit mode
11	D4	D10	Data bit 4
12	D5	D9	Data bit 5
13	D6	D8	Data bit 6
14	D7	D7	Data bit 7

The LCD uses 4-bit mode. The Arduino LiquidCrystal constructor must match this exact order: LiquidCrystal lcd(12, 11, 10, 9, 8, 7);

6.2 ACS712 current sensor connections

ACS712 pin	Signal	Connection
1 and 2	IP+	Current path from the panel/source side
3 and 4	IP−	Current path toward the load side
5	GND	Common ground
6	FILTER	Leave open for basic simulation; optional filter capacitor
7	VIOUT	Arduino A1
8	VCC	Arduino +5 V

The IP+ and IP− terminals form the current-carrying path and must be in series with the load. VIOUT is a separate low-voltage signal connection to A1. Follow the pin labels on the Proteus component because the symbol orientation may differ from a physical package drawing.

6.3 Voltage divider connections

Part / node	Connection
R1	4 kΩ; upper terminal to solar-panel positive measurement node
R2	1 kΩ; lower terminal to common ground
R1–R2 junction	Arduino A0
Divider ground	Solar-panel negative / measurement reference ground

The divider midpoint must be connected to A0. Do not connect the full panel voltage directly to A0.

6.4 Common power and ground

Component pin	Connection
Arduino 5V	5 V supply rail
LCD VDD (pin 2)	5 V supply rail
ACS712 VCC (pin 8)	5 V supply rail
Arduino GND	Common ground
LCD VSS (pin 1) and RW (pin 5)	Common ground
ACS712 GND (pin 5)	Common ground
Voltage-divider lower end	Common measurement ground

All low-voltage measurement circuits must share a common reference ground. Ensure that the Proteus VCC terminal is actually connected to the Arduino 5 V net; a power terminal label alone does not guarantee the intended connection if the nets are separate.

7. Arduino Source Code

The following sketch is written for the pin assignments in this manual. It displays a welcome message for four seconds, a startup message for two seconds, and then continuously displays voltage, current, and power.

```
#include <LiquidCrystal.h>

// LCD: RS, E, D4, D5, D6, D7
LiquidCrystal lcd(12, 11, 10, 9, 8, 7);

// Analog sensor pins
const int voltagePin = A0;
const int currentPin = A1;

// Voltage divider resistors
const float R1 = 4000.0;
const float R2 = 1000.0;

// ACS712 5A sensitivity
const float sensitivity = 0.185;  // V/A

// Nominal ACS712 zero-current output
const float zeroCurrentVoltage = 2.5;

// Arduino ADC reference voltage
const float referenceVoltage = 5.0;

void setup() {
  lcd.begin(20, 4);

  // Welcome screen
  lcd.setCursor(5, 0);
  lcd.print("WELCOME TO");

  lcd.setCursor(6, 1);
  lcd.print("FEC CSE");

  lcd.setCursor(0, 2);
  lcd.print("MICROCONTROLLER LAB");

  lcd.setCursor(4, 3);
  lcd.print("SOLAR PROJECT");

  delay(4000);

  lcd.clear();
  lcd.setCursor(1, 1);
  lcd.print("Solar Monitoring");
  lcd.setCursor(3, 2);
  lcd.print("System Starting");

  delay(2000);
  lcd.clear();
}

void loop() {
  // 1. Read and average panel-voltage samples
  long voltageTotal = 0;
  for (int i = 0; i < 100; i++) {
    voltageTotal += analogRead(voltagePin);
    delay(2);
  }

  float voltageReading = voltageTotal / 100.0;
  float sensorVoltage =
      voltageReading * referenceVoltage / 1023.0;
```

```
  // Convert divider output to panel voltage
  float solarVoltage =
     sensorVoltage * (R1 + R2) / R2;

  // 2. Read and average ACS712 samples
  long currentTotal = 0;
  for (int i = 0; i < 100; i++) {
    currentTotal += analogRead(currentPin);
    delay(2);
  }

  float currentReading = currentTotal / 100.0;
  float currentSensorVoltage =
     currentReading * referenceVoltage / 1023.0;

  // Convert ACS712 output voltage to current
  float solarCurrent =
     (currentSensorVoltage - zeroCurrentVoltage)
     / sensitivity;

  // Display magnitude only; direction is not shown
  solarCurrent = abs(solarCurrent);

  // 3. Calculate electrical power
  float solarPower = solarVoltage * solarCurrent;

  // 4. Display values on the 20x4 LCD
  lcd.setCursor(0, 0);
  lcd.print("FEC CSE SOLAR SYSTEM");

  lcd.setCursor(0, 1);
  lcd.print("Voltage : ");
  lcd.print(solarVoltage, 2);
  lcd.print(" V     ");

  lcd.setCursor(0, 2);
  lcd.print("Current : ");
  lcd.print(solarCurrent, 2);
  lcd.print(" A     ");

  lcd.setCursor(0, 3);
  lcd.print("Power   : ");
  lcd.print(solarPower, 2);
  lcd.print(" W     ");

  delay(1000);
}
```

8. Expected LCD Output

Welcome screen (shown for approximately four seconds):

WELCOME TO
FEC CSE
MICROCONTROLLER LAB
SOLAR PROJECT

Monitoring screen (illustrative readings only):

FEC CSE SOLAR SYSTEM
Voltage : 12.00 V

Current : 1.00 A
Power : 12.00 W

Actual readings depend on the Proteus solar-panel model, load, sensor offset, and supply/reference assumptions.

9. Sample Mathematical Calculation

Assume the panel voltage is 12 V and the load current is 1 A.

Voltage divider: VA0 = 12 × 1 kΩ / (4 kΩ + 1 kΩ) = 2.4 V.

Arduino reconstructs panel voltage: VPANEL = 2.4 × 5 = 12 V.

ACS712 example: if VIOUT = 2.685 V and VZERO = 2.5 V, then I = (2.685 − 2.5) / 0.185 = 1 A.

Power: P = V × I = 12 × 1 = 12 W.

10. Proteus Simulation Procedure

1. Open Proteus Design Suite and create a new schematic project.
2. Press P (Pick Devices) and add ARDUINO UNO, ACS712ELCTR-05B-T, LM044L, resistors, solar-panel model, load, and any meters used in the schematic.
3. Set R1 to 4 kΩ and R2 to 1 kΩ.
4. Wire the LCD according to Section 6.1. Confirm VSS is GND, VDD is +5 V, and VEE is connected for suitable contrast.
5. Connect the voltage-divider midpoint to A0 and ACS712 VIOUT to A1.
6. Place the ACS712 IP+ and IP− current path in series with the load.
7. In Arduino IDE, compile the sketch for Arduino UNO. Enable verbose output during compilation if needed to locate the generated HEX file.
8. Double-click the Arduino UNO in Proteus and browse to the compiled HEX file. Set the clock frequency to 16 MHz.
9. Check all power and ground nets, then start the simulation.
10. Observe the welcome screen followed by the live voltage, current, and power display.

11. Observation Table

Trial	Voltage (V)	Current (A)	Power (W)
1			
2			
3			
4			
5			

For each trial, calculate power using P = V × I and compare the calculated value with the LCD reading.

12. Precautions and Troubleshooting

- Never connect the full solar-panel voltage directly to Arduino A0. Use the voltage divider and ensure A0 remains within the allowed input range.
- Connect the ACS712 current path in series with the load; do not connect IP+ and IP− as signal pins.
- Connect ACS712 VIOUT to A1, VCC to 5 V, and GND to common ground.
- Use the correct ACS712 sensitivity: 0.185 V/A for the 5 A version.
- Calibrate the zero-current output. The nominal 2.5 V value may not match the exact simulated sensor model.
- For the LCD, connect pin 1 (VSS) to GND, pin 2 (VDD) to +5 V, and pin 3 (VEE/VO) to GND or a contrast potentiometer.

- Connect LCD RW (pin 5) to GND for write-only operation.
- Check the LCD data-pin order against the LiquidCrystal constructor.
- Ensure the Arduino and sensors share the intended common ground and 5 V supply.
- Do not power a motor directly from an Arduino GPIO pin. Use an appropriately rated motor driver or switching circuit and suitable protection.

13. Viva Questions

Q. What is the role of Arduino UNO in this project?

Answer: It reads sensor signals, converts them into useful values, calculates power, and controls the LCD.

Q. Why is a voltage divider required?

Answer: It reduces the panel voltage to a safe level for the Arduino analog input.

Q. What is the ACS712 used for?

Answer: It measures current using the Hall effect and provides an analog voltage output.

Q. What is the nominal sensitivity of the ACS712 5 A version?

Answer: Approximately 185 mV/A or 0.185 V/A.

Q. Why is the ACS712 zero-current output approximately 2.5 V?

Answer: Its output is biased around half of the 5 V supply so it can represent current in either direction.

Q. What is the formula for electrical power?

Answer: $P = V \times I$.

Q. Why use a 20×4 LCD?

Answer: It provides four rows of 20 characters, allowing several parameters to be shown simultaneously.

Q. What does analogRead() return on an Arduino UNO?

Answer: A 10-bit ADC reading from 0 to 1023.

Q. What happens if 12 V is connected directly to A0?

Answer: It exceeds the normal input range and may damage the microcontroller.

Q. Why is common ground important?

Answer: It provides a shared voltage reference for the connected circuits.

14. Student Assignment

11. Modify the LCD welcome message to include your lab group or student ID.
12. Display the voltage, current, and power values with two decimal places.
13. Change the voltage-divider resistor values and update the calculation in the code.
14. Explain why the ACS712 must be connected in series with the load.
15. Explain how the Arduino converts an analog sensor voltage into a digital reading.
16. Record five sets of voltage and current readings and calculate the corresponding power.

15. Conclusion

An Arduino-based solar monitoring system was designed and prepared for simulation using Proteus Design Suite. The voltage divider measures the solar-panel voltage, the ACS712 measures load current, and Arduino UNO calculates electrical power. The LM044L 20×4 LCD displays the welcome message and measured parameters. The experiment demonstrates microcontroller programming, analog interfacing, sensor integration, LCD interfacing, and basic embedded-system design.

Instructor Note: Verify the Final Schematic

The pin tables and source code in this manual define the intended wiring. Before conducting the lab, compare every wire in the Proteus schematic against the tables. In particular, verify the LCD power/contrast pins, the Arduino 5 V rail, the common ground, the voltage-divider midpoint at A0, and the ACS712 VIOUT connection at A1. Also confirm that the ACS712 current path is actually in series with the load.

The current-sensor example assumes a unidirectional current magnitude display and a nominal zero-current offset. A real or more detailed simulation may require calibration and a suitable current-sampling method.